

NAWIGACJA ROBOTÓW HUMANOIDALNYCH

WYKŁAD 4

Chód, stabilność, kontrola
i integracja pełnego systemu

dr inż. Mateusz Pomianek



Stabilność i lokomocja

Kontrola wykonania

Architektura ROS 2

Bezpieczeństwo

Plan wykładu

Sekcja 1

Stabilność i lokomocja: ZMP, Capture Point, LIPM, chód statyczny i dynamiczny, zakłócenia

Sekcja 2

Kontrola wykonania ruchu: warstwy sterowania, WBC, PID, śledzenie trajektorii, symulacja

Sekcja 3

Architektura ROS 2, bezpieczeństwo, walidacja eksperymentalna, studium przypadku

Kody Python

ZMP, LIPM, PID, saturacja sygnału, pętla symulacji, komunikacja ROS 2

01

SEKCJA

Stabilność i lokomocja

*Chód, ZMP, Capture Point, LIPM i reakcja na
zakłócenia*



Pełny pipeline: od percepcji do postawienia kroku

1

Percepcja

IMU, LiDAR, RGB-D,
enkodery – fuzja
sensoryczna i
estymacja stanu

2

Planowanie

A* + DWA: ścieżka
globalna i lokalna
przekazana do
planisty footstep

3

Generacja kroków

Sekwencja stóp,
trajektoria CoM z
modelu LIPM,
weryfikacja ZMP

4

Wykonanie

WBC + PID: komendy
momentów do
aktuatorów, odczyt
stanu i zamknięcie
pętli

Chód statyczny a chód dynamiczny

Chód statyczny

- CoM zawsze nad wielobokiem podparcia
- Możliwy przy bardzo niskich prędkościach
- Margines stabilności statycznej zawsze >0
- Prosta implementacja – brak dynamiki LIPM
- Zastosowanie: schody, nierówny teren, precyzja
- Wada: wolny, nienaturalny, energochłonny

Chód dynamiczny

- CoM może chwilowo wyjść za wielobok podparcia
- Wymagana kontrola ZMP w czasie rzeczywistym
- Naturalny, energooszczędny marsz człowieka
- Modele: LIPM, 3D-LIPM, Spring-Mass
- Zastosowanie: szybkie przemieszczanie, biuro
- Wada: wymaga pełnego WBC i szybkiego planisty

Środek masy i wielobok podparcia

- **Wielobok podparcia** (support polygon): wypukłe otoczek punktów kontaktu stóp z podłożem
- **Stabilność statyczna**: rzut CoM pionowo w dół musi leżeć wewnątrz wieloboku podparcia
- **Faza jednostronnego podparcia**: wielobok = tylko jeden prostokąt stopy (~20×9 cm)
- **Faza podwójnego podparcia**: wielobok = suma obu stóp + przestrzeń między nimi
- **Margines stabilności** (stability margin): minimalna odległość rzutu CoM od krawędzi wieloboku
- **Cel**: utrzymać margines $\geq d_{\text{safe}} = 2\text{--}3\text{ cm}$ przez cały cykl kroku
- **Monitorowanie**: obliczane w każdym cyklu sterowania (1 kHz) na podstawie stanu stawów

Zero Moment Point - definicja i znaczenie

- **ZMP** to punkt na podłożu, w którym suma momentów sił kontaktu jest zerowa
- **Warunek stabilności dynamicznej:** ZMP musi leżeć wewnątrz wieloboku podparcia
- **Obliczanie ZMP z trajektorii CoM (LIPM):** $ZMP = x_CoM - (z_CoM/g) \cdot \ddot{x}_CoM$
- **Praktyczna interpretacja:** ZMP dąży do stopy podporowej podczas przenoszenia ciężaru
- **Margines ZMP:** typowo 2–4 cm od krawędzi; w trudnym terenie 5–8 cm
- **Planowanie oparte na ZMP:** wyznacz dozwolony ZMP → oblicz trajektorię CoM wstecznie
- **Biblioteki:** Choreonoid, OpenHRP3, DART – wbudowane generatory trajektorii ZMP/CoM

Capture Point i Linear Inverted Pendulum Model

Capture Point (CP)

- CP = punkt na podłożu, na który należy postawić stopę, by zatrzymać upadek
- $CP = x_{CoM} + (1/\omega_0) \cdot \dot{x}_{CoM}$
- $\omega_0 = \sqrt{g / z_{CoM}}$ – częstość LIPM
- Jeśli CP poza zasięgiem kroku → robot upadnie
- Zastosowanie: reakcja na pchnięcia, korekty równowagi
- Narzędzie: push-recovery reflex

LIPM - uproszczona dynamika chodu

- Ciało traktowane jako punkt masowy (CoM) na sztywnym, bezmasowym wahadle
- Ruch CoM w 2D: $\ddot{x} = \omega_0^2 \cdot (x - z_{ZMP})$
- Rozwiązanie: $x(t) = C_1 \cdot \cosh(\omega_0 t) + C_2 \cdot \sinh(\omega_0 t) + x_{ZMP}$
- Generowanie trajektorii CoM przez optymalizację QP
- Podstawa większości komercyjnych humanoida (ASIMO, Atlas, NAO)
- Ograniczenie: zakłada stałą wysokość CoM

Obliczanie ZMP z trajektorii CoM (model LIPM)

```
import numpy as np

def compute_zmp(com_pos: np.ndarray,
               com_acc: np.ndarray,
               com_height: float,
               g: float = 9.81) -> np.ndarray:
    """Wyznacza ZMP z przyspieszenia CoM (model LIPM, 2D: x i y)."""
    # com_pos [m], com_acc [m/s^2], com_height [m]
    zmp_x = com_pos[0] - (com_height / g) * com_acc[0]
    zmp_y = com_pos[1] - (com_height / g) * com_acc[1]
    return np.array([zmp_x, zmp_y])

# Przykład:
pos = np.array([0.05, 0.0]) # CoM 5 cm przed srodkiem stopy
acc = np.array([0.3, 0.0]) # przyspieszenie [m/s^2]
zmp = compute_zmp(pos, acc, com_height=0.85)
print(f'ZMP = {zmp}')      # np. [0.024, 0.0] m
```

Opis: Wynik ZMP [m] należy porównać z wielobokiem podparcia i sprawdzić margines bezpieczeństwa

Prosty model LIPM - symulacja ruchu CoM

```
import numpy as np

class LIPM:
    def __init__(self, com_height: float = 0.85, g: float = 9.81):
        self.omega = np.sqrt(g / com_height) # czestotliwosc LIPM [rad/s]
        self.x = 0.0 # pozycja CoM [m]
        self.vx = 0.0 # predkosc CoM [m/s]

    def step(self, zmp_x: float, dt: float) -> tuple:
        """Jeden krok calkowania rownania LIPM:  $x_{ddot} = \omega^2(x - zmp)$ ."""
        acc = self.omega**2 * (self.x - zmp_x)
        self.vx += acc * dt
        self.x += self.vx * dt
        return self.x, self.vx

lipm = LIPM(com_height=0.85)
for t in np.arange(0, 1.0, 0.005):
    x, vx = lipm.step(zmp_x=0.02, dt=0.005) # ZMP 2 cm przed stopa
```

Opis: $\omega = \sqrt{g/h} \approx 3,4$ rad/s dla $h=0,85$ m; numeryczne całkowanie Euler – wystarczające dla demonstracji.

Reakcja na zakłócenia zewnętrzne

- Pchnięcie (push): nagła zmiana prędkości CoM → CP wychodzi z dozwolonej strefy → korekta kroku
- Mechanizm ankle strategy: regulacja momentu w stawie skokowym (małe zakłócenia $\leq \sim 5 \text{ N}\cdot\text{s}$)
- Mechanizm hip strategy: ruch tułowia kompensuje przesunięcie CP (średnie zakłócenia)
- Mechanizm stepping strategy: postawienie dodatkowego kroku w kierunku CP (duże zakłócenia)
- Poślizg stopy: detekcja z czujnika F/T – nagły skok siły bocznej → reset estymatora i replan
- Utrata kontaktu: siła normalna $F_z < 10 \text{ N}$ → przejście do trybu push-recovery
- Opóźniona detekcja przeszkody: aktywacja hamowania awaryjnego i replan lokalny (10 ms)

Stabilność, a decyzje nawigacyjne

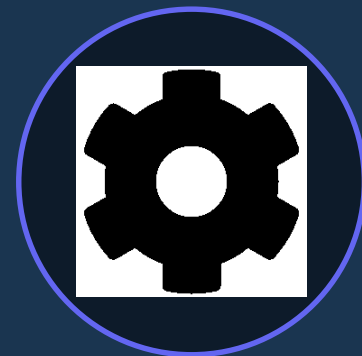
- Reguła nadrzędna: decyzja o ruchu jest akceptowana tylko wtedy, gdy predykcja ZMP pozostaje w wieloboku
- Sprzężenie planista–kontroler: planista kroków przekazuje cele, kontroler zgłasza aktualny margines
- Prędkość chodu a stabilność: wyższa prędkość = mniejszy czas podwójnego podparcia = wyższe ryzyko
- Teren pochyły lub śliski: kontroler redukuje prędkość i wydłuża fazę podwójnego podparcia
- Sygnał margines stabilności: gdy $SM < \text{threshold}$ → planista obniża prędkość lub zatrzymuje robota
- Priorytet: stabilność > plan ruchu > optymalizacja energii > czas misji
- Architektura: stability monitor działa w pętli 1 kHz i może vetować komendy planisty lokalnego

02

SEKCJA

Kontrola wykonania ruchu

Warstwy sterowania, WBC, PID, symulacja, akulatory



Warstwy sterowania w humanoidzie

Warstwa wysoka (High-level)

- Planista globalny i lokalny
- Generator sekwencji kroków
- Trajektoria CoM (LIPM / QP)
- Częstość: 10–50 Hz
- Wyjście: ref. trajektoria CoM, stóp

Warstwa środkowa (WBC)

- Whole-Body Controller
- Optymalizacja całego ciała
- Przestrzeganie kontaktów i ZMP
- Częstość: 200–500 Hz
- Wyjście: ref. kąty stawów lub τ_{ref}

Warstwa niska (aktuator)

- Regulatory PID / impedancji
- Pętle momentowe / prędkościowe
- Enkodery i czujniki F/T
- Częstość: 1000–5000 Hz
- Wyjście: napięcie / prąd silnika

Priorytety zadań w sterowaniu (WBC)

P1 Stabilność

ZMP w wieloboku podparcia; Capture Point w zasięgu; stabilność LIPM

P2 Kontakt stóp

Siły kontaktu ≥ 0 ; przyczepność (tarcie); poprawna orientacja stopy

P3 Realizacja ruchu

Śledzenie trajektorii CoM i stóp zgodnie z planem footstep

P4 Pozycja tułowia

Orientacja tułowia do pionu;
pozycja głowy do percepcji

P5 Minimalizacja

Minimalizacja zużycia energii
i momentów stawowych
(*regularyzacja*)

Zasada działania Whole-Body Control (WBC)

- **WBC** wyznacza optymalny wektor momentów τ dla wszystkich stawów jednocześnie
- **Formułowane jako QP** (Quadratic Programming): $\min \frac{1}{2} \cdot \tau^T Q \tau$ s.t. $A\tau \leq b$
- **Ograniczenia**: równania dynamiki, siły kontaktu, zasięgi stawów, ograniczenia ZMP
- **Hierarchia zadań**: każde zadanie ma macierz wagową; zadania niższe nie zakłócają wyższych
- **Null-space projection**: niższe priorytety realizowane w przestrzeni zerowej wyższych
- **Czas obliczeń**: solver OSQP lub qpOASES wyznacza rozwiązanie w ~ 1 ms na CPU embedded
- **Integracja z LIPM**: trajektoria CoM z modelu LIPM $\rightarrow$ WBC $\rightarrow$ momenty stawów

Regulator PID dla kierunku yaw robota

```
class PIDController:
    def __init__(self, kp, ki, kd, output_limit=1.0):
        self.kp, self.ki, self.kd = kp, ki, kd
        self.limit = output_limit
        self.integral = 0.0
        self.prev_error = 0.0

    def update(self, setpoint: float, measured: float, dt: float) -> float:
        """Krok regulatora PID; zwraca sygnał sterujący [rad/s]."""
        error = setpoint - measured
        # Normalizacja błędu kąta do [-pi, pi]
        error = (error + 3.14159) % (2*3.14159) - 3.14159
        self.integral += error * dt
        derivative = (error - self.prev_error) / max(dt, 1e-6)
        self.prev_error = error
        output = self.kp*error + self.ki*self.integral + self.kd*derivative
        return max(-self.limit, min(self.limit, output)) # saturacja
```

Opis: $K_p \approx 2,0$; $K_i \approx 0,1$; $K_d \approx 0,05$ – typowe wartości dla regulacji kierunku humanoida z IMU 200 Hz

Śledzenie trajektorii - regulator nadążny

- **Cel:** robot ma podążać za zadaną trajektorią ($x_{ref}(t)$, $y_{ref}(t)$, $yaw_{ref}(t)$) z minimalnym błędem
- **Błąd boczny** (cross-track error): odległość prostopadła od aktualnej pozycji do trajektorii
- **Błąd podłużny** (along-track error): opóźnienie lub wyprzedzenie względem parametru t
- **Metoda Pure Pursuit:** cel = punkt na trajektorii w odległości look-ahead L_d od robota
- **Metoda Stanley Controller:** kombinacja błędu bocznego i kąta nagłówka – stosowana w autonomicznych pojazdach
- **Dla humanoida:** oddzielne regulatory dla CoM (translacja) i tułowia (orientacja yaw)
- **Anti-windup:** ograniczenie całkowania w PID, gdy robot jest poza pętlą (np. stoi w miejscu)

Ograniczenia aktuatorów i ich wpływ na planowanie

- **Moment maksymalny $\tau_{\max}$:** przekroczenie powoduje przegrzanie silnika lub uszkodzenie przekładni
- **Prędkość kątowa $\omega_{\max}$:** zbyt szybki ruch może wywołać efekt kinetyczny zaburzający stabilność
- **Saturacja sygnału:** gdy komendy WBC przekraczają możliwości, planista musi zwolnić
- **Przegrzewanie termiczne:** ciągła praca przy $\tau_{\max}$ skraca żywotność; monitoring temperatury uzwojeń
- **Pasma przenoszenia:** aktuator serwo działa poprawnie do ~ 50 Hz; powyżej – opóźnienie fazowe
- **Wpływ na planowanie:** planista globalny może obniżyć prędkość tras o 20% przy dużym obciążeniu
- **Narzędzia diagnostyczne:** logowanie momentów na osi czasu pozwala identyfikować przeciążone stawy

Saturacja i monitorowanie sygnału sterującego

```
import numpy as np

def saturate_torques(tau_desired: np.ndarray,
                    tau_max: np.ndarray,
                    warn_ratio: float = 0.85) -> tuple:
    """Nasycenie momentow stawowych z ostrzezeniem o przeciazeniu."""
    tau_sat = np.clip(tau_desired, -tau_max, tau_max)
    sat_ratio = np.abs(tau_desired) / (tau_max + 1e-6)
    overloaded = np.where(sat_ratio > warn_ratio)[0]
    warnings = []
    for idx in overloaded:
        warnings.append(f'Staw {idx}: {sat_ratio[idx]*100:.1f}% tau_max')
    return tau_sat, warnings

tau_ref = np.array([120.0, -85.0, 200.0]) # zadane momenty [Nm]
tau_lim = np.array([100.0, 100.0, 100.0]) # limity
tau_out, warns = saturate_torques(tau_ref, tau_lim)
```

Opis: Funkcja clipuje momenty i generuje ostrzeżenia gdy staw > 85% limitu – sygnał do zwolnienia planisty.

Symulacja przed testami na robocie

- **Cyfrowy bliźniak (digital twin):** model robota z dokładnymi parametrami inercji, tarcia i aktuatorów
- **Gazebo (ROS 2):** popularny w akademii; silniki fizyki: ODE, Bullet, DART; integracja z ROS 2 control
- **MuJoCo:** lider w lokomocji RL; dokładna symulacja kontaktu; używany przez Boston Dynamics, DeepMind
- **PyBullet:** szybki, pythonowy; prostszy w konfiguracji; stosowany w badaniach akademickich
- **Zasada bezpiecznego przejścia:** algorytm działa poprawnie w symulacji → testy na robocie w ograniczonej przestrzeni
- **Metryki sym2real gap:** różnica prędkości kroku, błędu ZMP, zużycia energii między symulacją a robotem
- **Domain randomization:** losowanie tarcia, masy i opóźnień w symulacji wzmacnia odporność kontrolera

Pętla sterowania w symulacji (*PyBullet* / *abstrakcja*)

```
import numpy as np

class SimulationLoop:
    def __init__(self, robot, planner, controller, dt=0.005):
        self.robot      = robot      # interfejs do symulatora
        self.planner     = planner    # planista (ścieżka + footstep)
        self.controller  = controller # WBC lub PID
        self.dt          = dt

    def run(self, goal_pos, max_steps=10_000):
        for step in range(max_steps):
            state      = self.robot.get_state()      # odczyt stanu
            footsteps  = self.planner.plan(state, goal_pos)
            tau        = self.controller.compute(state, footsteps)
            tau, _     = saturate_torques(tau, self.robot.tau_max)
            self.robot.apply_torques(tau)            # wyslij komendy
            self.robot.step_simulation(self.dt)
            if np.linalg.norm(state['pos'] - goal_pos) < 0.05: break
```

Opis: Pętla odczyt→plan→kontrola→aktuacja w kroku $dt=5$ ms (200 Hz); kryterium stopu: odległość < 5 cm

03

SEKCJA

Architektura systemowa, bezpieczeństwo i walidacja

*ROS 2, fail-safe, monitoring jakości i studium
przypadku*



Integracja modułów w ROS 2

- **Topiki (*topics*):** asynchroniczna wymiana danych; sensor_msgs/Imu, nav_msgs/Odometry, geometry_msgs/Twist
- **Serwisy (*services*):** synchroniczne wywołania; np. /set_goal, /reset_localization, /emergency_stop
- **Akcje (*actions*):** długotrwałe zadania z feedbackiem; nav2 NavigateToPose – standard dla nawigacji
- **TF2:** drzewo transformacji; każdy node publikuje tf; lookupTransform() z buforowaniem przez ~5 s
- **ros2_control:** ustandaryzowany interfejs dla aktuatorów – hardware_interface + controller_manager
- **QoS (*Quality of Service*):** RELIABLE dla danych krytycznych (stan robota), BEST_EFFORT dla sensorów
- **Modularność:** każdy moduł (SLAM, planner, WBC) jako oddzielny node lub lifecycle node

Przykładowy pipeline uruchomieniowy (ROS 2)

1. Sensory

lidar_driver, imu_driver,
depth_camera_node – odczyt
i preprocessing danych sensorycznych

2. Lokalizacja

slam_toolbox lub Nav2 AMCL – estymacja
pozycji w mapie; publikacja /odom
i /map→base tf

3. Mapa kosztów

Nav2 costmap_2d – global_costmap
i local_costmap aktualizowane z LiDAR
i RGB-D

4. Planista globalny

Nav2 NavFn lub Smac Planner – A* lub
Hybrid-A* z cost map; ścieżka do /plan
topic

5. Planista lokalny

Nav2 DWB Controller – DWA z krytykami
trajektorii; komendy do /cmd_vel lub
footstep

6. WBC + wykonanie

ros2_control + whole_body_controller –
śledzenie footstep, moment stawowy,
ZMP monitoring

Publikacja celu i odbiór stanu w ROS 2 (Python)

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import PoseStamped
from nav_msgs.msg import Odometry

class NavClient(Node):
    def __init__(self):
        super().__init__('nav_client')
        self.goal_pub = self.create_publisher(
            PoseStamped, '/goal_pose', 10)
        self.odom_sub = self.create_subscription(
            Odometry, '/odom',
            self.odom_callback, 10)
    def odom_callback(self, msg: Odometry):
        x = msg.pose.pose.position.x
        y = msg.pose.pose.position.y
        self.get_logger().info(f'Pos: ({x:.2f}, {y:.2f})')
    def send_goal(self, x, y):
        msg = PoseStamped()
        msg.header.frame_id = 'map'
        msg.pose.position.x = x; msg.pose.position.y = y
        self.goal_pub.publish(msg)
```

Opis: Minimalna integracja ROS 2: subscriber /odom + publisher /goal_pose; Nav2 obsługuje resztę pipeline'u.

Bezpieczeństwo i mechanizmy fail-safe

Detekcja sytuacji awaryjnych

- Detekcja upadku: akcelerometr $|a| > 3g$ lub roll/pitch $> 45^\circ$
- Brak kontaktu obu stóp: $F_{z_L} + F_{z_R} < 20\text{ N}$ przez $>200\text{ ms}$
- Przekroczenie limitu momentu: $\tau > 1,2 \cdot \tau_{\text{max}}$ przez $>50\text{ ms}$
- Utrata sensorów: brak wiadomości IMU przez $>50\text{ ms} \rightarrow \text{alarm}$
- Kolizja: siła na strukturze $> F_{\text{collision_thresh}}$
- Błąd lokalizacji: kowariancja $P > P_{\text{max}} \rightarrow \text{stop i relokalizacja}$

Reakcje bezpieczne (fail-safe)

- Zatrzymanie awaryjne: momenty $\rightarrow 0$ w ciągu 50 ms
- Bezpieczna postura: zejście do pozycji kucającej lub siadania
- Tryb degradacji: nawigacja tylko na IMU + enkodery
- Powiadomienie operatora: alert przez ROS 2 diagnostics
- Automatyczny restart po relokalizacji (autorecovery)
- Hardware e-stop: przycisk fizyczny odcina zasilanie aktuatorów

Metryki online do monitorowania jakości nawigacji

- **Błąd lokalizacji:** bieżące RMSE pozycji względem trajektorii referencyjnej [m]
- **Liczba korekt kroku:** ile razy planista footstep przeplanował z powodu zmiany otoczenia
- **Margines ZMP:** $\min(\text{distance}(\text{ZMP}, \text{edge}))$ w bieżącym cyklu [cm]
- **Temperatura aktuatorów:** topik `/joint_temperatures`; próg ostrzegawczy 70°C, krytyczny 85°C
- **Straty kontaktu:** ile razy $F_z < \text{próg}$ podczas fazy podparcia – wskaźnik poślizgu lub złej nawierzchni
- **Near-miss:** liczba sytuacji gdy ZMP < 1 cm od krawędzi wieloboku w ostatnich 60 s
- **Narzędzie:** ROS 2 diagnostics + `rqt_runtime_monitor`; zapis do MCAP/rosbag2 do późniejszej analizy

Walidacja eksperymentalna - scenariusze testowe

Nawigacja prosta

Przejsie A→B 10 m po równej powierzchni; metryka: RMSE pozycji, czas, zużycie energii

Omijanie przeszkód

5 statycznych + 2 dynamiczne przeszkody; metryka: liczba kolizji, liczba replanu

Schody

3 stopnie $h=15$ cm; metryka: powodzenie, czas, liczba korekt ZMP, szczyt momentów

Pchnięcie boczne

Impuls 20 Ns; metryka: amplituda CP, czas odzyskania równowagi, krok korekcyjny

Drzwi i progi

Otwieranie klamki, przekroczenie progu 3 cm; metryka: sukces, czas interakcji

Długa misja

45 min patrol budynku; metryka: dryft SLAM, zużycie energii, liczba incydentów

Najczęstsze błędy projektowe

- **Zbyt optymistyczny model dynamiki:** symulator nie uwzględnia ugięcia struktury ani tarcia statycznego stawów
- **Brak synchronizacji czasowej:** IMU i kamera nie zsynchronizowane → duchy w mapie i błędne fusje
- **Ignorowanie opóźnień:** kamera ma 33 ms opóźnienie; bez kompensacji EKF daje błędną pozycję
- **Niestabilność kontaktu:** zbyt twarde regulatory powodują drgania w fazie kontaktu stopy z podłożem
- **Brak anti-windup w PID:** przy saturacji moment integrujący rośnie bez ograniczeń → overshooting
- **Monolityczna architektura:** cały system w jednym procesie – awaria jednego modułu blokuje całość
- **Zbyt rzadkie testy na robocie:** przeniesienie działającego symulatora na fizyczny robot bez etapów pośrednich

Studium przypadku: humanoid w zadaniu usługowym

- **Scenariusz:** robot dostarcza dokument z recepcji (parter) do sali konferencyjnej (2. piętro)
- **Etap 1 – Inicjalizacja:** wczytanie mapy budynku, lokalizacja AMCL, plan globalny z windą lub schodami
- **Etap 2 – Korytarz parter:** DWA z detekcją YOLO, 3 osoby omijane, ORCA dla dynamicznych obiektów
- **Etap 3 – Windy:** detekcja drzwi, wciśnięcie guzika (manipulacja), oczekiwanie na otwarcie, wejście
- **Etap 4 – Korytarz 2. piętro:** reinicjalizacja AMCL po wyjściu z windy, dojście do celu
- **Etap 5 – Interakcja:** podanie dokumentu, potwierdzenie odbioru, powrót trasy odwrotnej
- **Kluczowe wyzwania:** zachowanie w windzie (mała przestrzeń), interakcja z człowiekiem, niezawodność 45 min

PODSUMOWANIE

- 01 ZMP i Capture Point to fundamenty równowagi dynamicznej; planista CoM musi przestrzegać ich ograniczeń.
- 02 Whole-body control hierarchizuje zadania: stabilność > kontakt stóp > realizacja trajektorii.
- 03 PID, śledzenie trajektorii i saturacja aktuatorów to warunki bezpiecznego wykonania ruchu.
- 04 ROS 2 + TF2 integruje wszystkie moduły; synchronizacja i modularność są warunkiem niezawodności.
- 05 Walidacja w symulacji (MuJoCo, Gazebo) poprzedza testy na robocie – cyfrowy bliźniak oszczędza sprzęt.

PODSUMOWANIE CAŁEGO CYKLU

4 Wykładów z Nawigacji Robotów Humanoidalnych

W1

Percepcja
i mapowanie

W2

Lokalizacja
i fuzja sensoryczna

W3

Planowanie ruchu
i unikanie przeszkód

W4

Chód, stabilność
i integracja systemu

Pełna pętla nawigacyjna: Sensory → Estymacja stanu → Planowanie → Sterowanie → Aktuatory → Sensory

Reinforcement learning
dla lokomocji

Nawigacja
semantyczna

Social navigation
i HRI

Multimodalna
percepcja

Adaptacyjny
chód

Tematy dalszych badań i kierunki rozwoju

1. **Reinforcement learning dla lokomocji:** uczenie chodu end-to-end bez modelu (MuJoCo + PPO/SAC)
2. **Nawigacja semantyczna:** reprezentacje CLIP/ViLD; robot rozumie polecenia językowe i mapę semantyczną
3. **Social navigation:** modele ruchu tłumu, normy kulturowe, personalizacja interakcji człowiek–robot
4. **Multimodalna percepcja:** fuzja kamery, LiDAR, taktylna + duże modele wizyjno-językowe (VLM)
5. **Adaptacyjny chód:** zmiana wzorca chodu w zależności od terenu bez ręcznego programowania
6. **Whole-body manipulation:** jednoczesna nawigacja i manipulacja – otwieranie drzwi w ruchu
7. **Cloud robotics + edge AI:** ciężkie obliczenia SLAM i planowania na serwerze edge z niskim opóźnieniem